팀3 — 생산흐름 병목개선 · 납기지연 위험 예측

K-뉴딜 아카데미 스마트팩토리 캡스톤 · 3~6회차 통합 실습

데이터	분석단위	규모	기간
`team3_flow.csv`	생산로트 1건 (A→B→C→D 4공정을 거친 한 묶음)	1,600행 × 26열	2026년 7월 1일 ~ 10월 9일 (로트 시작시각 기준)

이 노트북에서 하는 일

데이터 탐색 → 시각화 → 전처리 → 머신러닝 모델 3종 → 성능평가 → 특성중요도

현장 상황

가상의 **소형모터 조립공장**입니다. 제품은 **A → B → C → D** 네 공정을 순서대로 거칩니다. 전체 주문량은 대체로 맞추지만, 일부 로트에서 **대기시간과 리드타임이 급증**하고 공정 사이에 **재공품(WIP)** 이 쌓이면서 납기지연이 발생합니다.

여기서 **병목(bottleneck)** 개념이 핵심입니다. 생산라인의 전체 처리량은 **가장 느린 공정 하나**가 결정합니다. 4개 공정이 각각 100개/시간, 100개/시간, 60개/시간, 100개/시간을 처리할 수 있다면 라인 전체는 **60개/시간**입니다. C공정 앞에는 물건이 쌓이고(WIP 증가), C공정 뒤 공정들은 일이 없어 기다립니다(아이들). 그래서 **병목이 아닌 공정에 인력을 투입하면 아무 효과가 없습니다.** 병목을 정확히 찾는 것이 개선의 출발점입니다.

라인 리더는 **교대 시작 전에** 데이터를 보고, 병목이 예상되는 공정에 인력을 지원하거나 작업순서를 조정합니다.

핵심 질문

> **어느 공정이 병목이고 지연의 주요 원인은 무엇인가? 교대 시작 전에 납기지연 위험 로트를 가려낼 수 있는가?**

판단 시점

> **교대 시작 전 (계획수량·제품군·투입 작업자 수·교체계획은 알고, 실제 사이클타임·대기시간·완료수량은 모름)**

판단 시점이 중요한 이유: **그 시점에 알 수 없는 값**을 입력으로 쓰면 시험 답안지를 미리 본 것과 같아집니다(**데이터 누출**). 아래 8절에서 직접 걸러냅니다.

---

1. 준비 — 라이브러리와 한글 폰트

도구	역할
`pandas` (pd)	표 데이터 처리 (엑셀에 해당)
`numpy` (np)	숫자 계산
`matplotlib` (plt) · `seaborn` (sns)	그래프
`scikit-learn`	머신러닝

아래 셀은 **한 번만** 실행하면 됩니다.

```python
# [준비 셀] 내용을 자세히 이해할 필요는 없습니다. 실행만 하세요.
# 라이브러리 불러오기 + 한글 폰트 설정
import os, sys, glob, warnings
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, classification_report, roc_curve,
                             roc_auc_score, recall_score, precision_score)
warnings.filterwarnings("ignore")
IN_COLAB = "google.colab" in sys.modules

# ------------------------------------------------------------------------
# 한글 폰트 설정
#   ※ 순서 주의: seaborn 스타일을 먼저 적용해야 한다.
#      sns.set_style() 이 글꼴 설정을 sans-serif 로 덮어쓰기 때문이다.
# ------------------------------------------------------------------------
sns.set_style("whitegrid")                      # ① 스타일 먼저

from matplotlib import font_manager as fm

KOREAN_FONTS = ["NanumGothic", "NanumBarunGothic", "Malgun Gothic", "AppleGothic",
                "Apple SD Gothic Neo", "Noto Sans CJK KR", "Noto Sans KR"]


def find_korean_font():
    """설치된 한글 폰트를 찾고, 없으면 Colab에서 나눔폰트를 설치한다."""
    have = {f.name for f in fm.fontManager.ttflist}
    for n in KOREAN_FONTS:
        if n in have:
            return n
    if IN_COLAB:                                # 나눔폰트 설치 후 다시 찾기
        print("한글 폰트를 설치하는 중입니다... (20초 정도 걸립니다)")
        os.system("apt-get install -y fonts-nanum > /dev/null 2>&1")
        os.system(f"{sys.executable} -m pip install -q koreanize-matplotlib")
        for path in glob.glob("/usr/share/fonts/truetype/nanum/Nanum*.ttf"):
            fm.fontManager.addfont(path)
        have = {f.name for f in fm.fontManager.ttflist}
        for n in KOREAN_FONTS:
            if n in have:
                return n
    return None

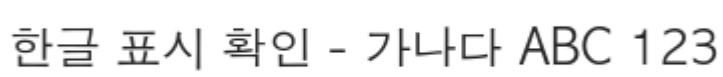

FONT = find_korean_font()                       # ② 폰트 나중에
if FONT:
    plt.rcParams["font.family"] = FONT
    plt.rcParams["font.sans-serif"] = [FONT] + list(plt.rcParams["font.sans-serif"])
plt.rcParams["axes.unicode_minus"] = False      # 음수 기호(-) 깨짐 방지
plt.rcParams["figure.figsize"] = (9, 4.5)
plt.rcParams["figure.dpi"] = 110

pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

print("준비 완료 | pandas", pd.__version__, "| Colab:", IN_COLAB)
print("한글 폰트 :", FONT if FONT else "찾지 못함")
```

```
# ③ 자가진단 — 아래 그림에 한글이 보이면 정상입니다
plt.figure(figsize=(5, 0.9))
plt.text(0.5, 0.5, "한글 표시 확인 — 가나다 ABC 123",
         ha="center", va="center", fontsize=15)
plt.axis("off")
plt.show()
print("위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.")
```

```
준비 완료 | pandas 2.2.3 | Colab: False
한글 폰트 : AppleGothic
```




```
위 그림의 글자가 □□□ 로 보이면 [런타임 > 런타임 다시 시작] 후 이 셀을 다시 실행하세요.
```

---

2. 데이터 가져오기

두 가지 방법이 있습니다. **① GitHub에서 바로 내려받기**(권장)가 안 되면 **② PC에서 업로드**를 쓰세요.

```
# ① GitHub에서 바로 내려받기
URL = "https://raw.githubusercontent.com/StillWork/capstone2610/main/team3_flow.csv"
df_raw = pd.read_csv(URL)

# ② PC에서 업로드 — ①이 안 되면 위 2줄을 지우고 아래 3줄의 주석을 푸세요
# from google.colab import files
# up = files.upload()
# df_raw = pd.read_csv(list(up.keys())[0], encoding="utf-8-sig")

print("크기(행, 열):", df_raw.shape)
df_raw.head(3)
```

```
크기(행, 열): (1600, 26)
```

	record_id	lot_start_time	shift	line_id	product_family	order_priority	planned_qty	actual_qty	available
0	FL-00001	2026-07-01 00:00:00	Day	L1	PF1	NORMAL	175	175	
1	FL-00002	2026-07-01 01:30:00	Night	L1	PF2	NORMAL	186	186	
2	FL-00003	2026-07-01 03:00:00	Day	L2	PF2	NORMAL	117	117	

---

3. 데이터 훑어보기

새 데이터를 받으면 **크기 → 자료형·결측 → 요약통계** 순으로 봅니다. 요약통계에서는 **불가능한 값**(음수 수량, 범위를 벗어난 값)이 있는지 확인합니다.

```
df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1600 entries, 0 to 1599
Data columns (total 26 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   record_id                1600 non-null   object 
 1   lot_start_time           1600 non-null   object 
 2   shift                    1600 non-null   object 
 3   line_id                  1600 non-null   object 
 4   product_family           1600 non-null   object 
 5   order_priority           1600 non-null   object 
 6   planned_qty              1600 non-null   int64  
 7   actual_qty               1600 non-null   int64  
 8   available_operators      1600 non-null   int64  
 9   changeover_min           1600 non-null   float64
 10  station_a_cycle_sec      1600 non-null   float64
 11  station_b_cycle_sec      1600 non-null   float64
 12  station_c_cycle_sec      1590 non-null   float64
 13  station_d_cycle_sec      1600 non-null   float64
 14  wait_before_b_min        1600 non-null   float64
 15  wait_before_c_min        1584 non-null   float64
 16  wait_before_d_min        1600 non-null   float64
 17  wip_start_qty            1600 non-null   int64  
 18  wip_end_qty              1600 non-null   int64  
 19  equipment_stop_min       1600 non-null   float64
 20  material_wait_min        1600 non-null   float64
 21  planned_lead_time_min    1600 non-null   float64
 22  actual_lead_time_min     1600 non-null   float64
 23  bottleneck_station       1600 non-null   object 
 24  on_time_completion_flag  1600 non-null   object 
 25  schedule_delay_min       1600 non-null   float64
dtypes: float64(13), int64(5), object(8)
memory usage: 325.1+ KB
```

```
display(df_raw.describe().T.round(2))

miss = df_raw.isna().sum()
print("[결측이 있는 변수]")
print(miss[miss > 0].to_string() if (miss > 0).any() else "없음")
```

	count	mean	std	min	25%	50%	75%	max
planned_qty	1,600.000	179.140	31.550	100.000	157.000	179.000	201.000	270.000
actual_qty	1,600.000	173.750	29.320	100.000	154.000	174.000	194.000	270.000
available_operators	1,600.000	8.560	1.480	5.000	8.000	9.000	10.000	12.000
changeover_min	1,600.000	5.080	10.320	0.000	0.000	0.000	0.000	42.600
station_a_cycle_sec	1,600.000	45.310	3.620	34.300	42.800	45.200	47.800	55.900
station_b_cycle_sec	1,600.000	48.960	4.220	34.900	46.100	48.900	51.700	67.200
station_c_cycle_sec	1,590.000	48.660	5.600	33.500	44.600	48.300	52.700	66.000
station_d_cycle_sec	1,600.000	43.080	2.980	32.900	41.100	43.100	45.000	53.800
wait_before_b_min	1,600.000	13.140	7.760	0.000	7.600	12.200	18.200	38.600
wait_before_c_min	1,584.000	11.940	8.160	0.000	6.200	10.900	16.700	44.800
wait_before_d_min	1,600.000	5.860	4.100	0.000	2.780	5.600	8.330	23.800
wip_start_qty	1,600.000	37.320	13.340	5.000	28.000	38.000	46.000	83.000
wip_end_qty	1,600.000	42.700	20.390	5.000	30.750	40.000	51.000	178.000
equipment_stop_min	1,600.000	3.230	7.950	0.000	0.000	0.000	0.000	49.200
material_wait_min	1,600.000	2.420	7.130	0.000	0.000	0.000	0.000	46.200
planned_lead_time_min	1,600.000	209.350	18.330	149.400	197.180	209.450	221.900	279.500
actual_lead_time_min	1,600.000	248.120	28.920	164.300	226.900	247.250	266.700	340.800
schedule_delay_min	1,600.000	38.970	21.960	0.000	23.180	37.300	52.500	132.800

```
[결측이 있는 변수]
station_c_cycle_sec    10
wait_before_c_min      16
```

---

4. 데이터 정제

항목	처리
C공정 사이클타임·대기시간 결측	**라인·제품군별 중앙값**으로 채움
이벤트 변수(설비정지·자재대기)	0이 많지만 **실제 사건**이므로 그대로 둠

```python
df = df_raw.copy()

# C공정 결측 → 같은 라인·제품군의 중앙값 (비슷한 조건끼리 묶어서 채운다)
for c in ["station_c_cycle_sec", "wait_before_c_min"]:
    n = int(df[c].isna().sum())
    df[c] = df[c].fillna(df.groupby(["line_id", "product_family"])[c].transform("median"))
    df[c] = df[c].fillna(df[c].median())
    print(f"{c} 결측 {n}건 → 라인·제품군별 중앙값으로 채움")

df["lot_start_time"] = pd.to_datetime(df["lot_start_time"])
print("\n남은 결측:", int(df.isna().sum().sum()), "건")
print("설비정지·자재대기는 0이 대부분이지만 실제 사건이므로 그대로 둔다.")
```

```
station_c_cycle_sec 결측 10건 → 라인·제품군별 중앙값으로 채움
wait_before_c_min 결측 16건 → 라인·제품군별 중앙값으로 채움

남은 결측: 0 건
설비정지·자재대기는 0이 대부분이지만 실제 사건이므로 그대로 둔다.
```

---

5. 목표변수 — 찾아내고 싶은 사건은 얼마나 드문가

목표변수는 **`on_time_completion_flag`** 입니다. 납기지연(N) 을 **1**, 나머지를 **0** 으로 둡니다.

양성이 드물면 **"전부 음성"이라고만 답해도 정확도가 매우 높습니다.** 그래서 이 데이터에서는 **정확도를 믿으면 안 됩니다.**

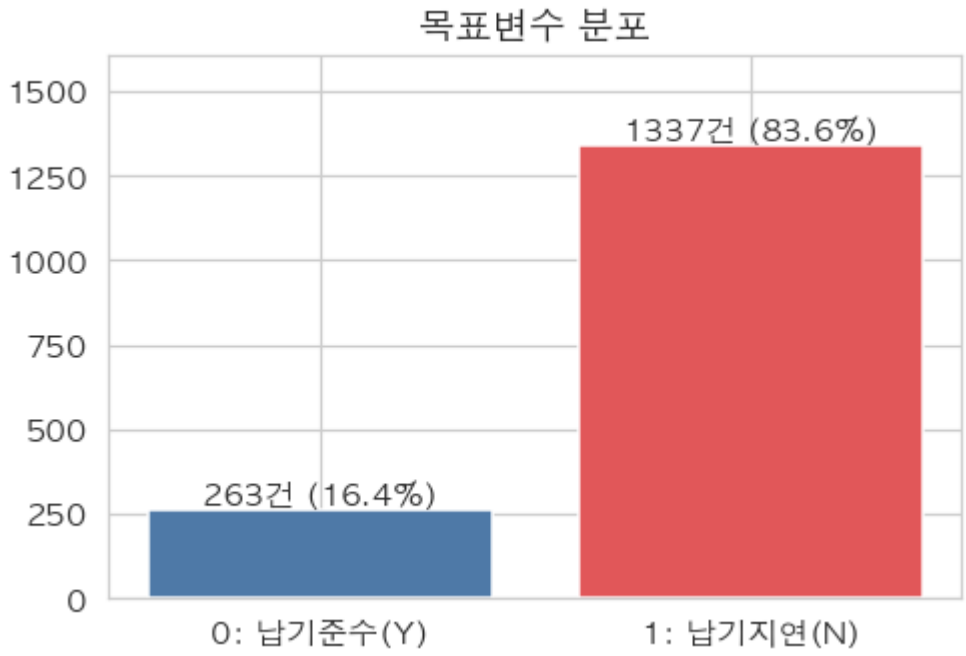
```python
y = (df['on_time_completion_flag'] == 'N').astype(int)          # 1 = 납기지연(N)
df["target"] = y

print(df["on_time_completion_flag"].value_counts().to_string())
print(f"\n양성(1) 비율            : {y.mean()*100:.2f}%")
print(f"'전부 음성' 모델의 정확도 : {(1-y.mean())*100:.2f}%  ← 이 숫자를 넘어야 의미가 있다")

cnt = y.value_counts().sort_index()
plt.figure(figsize=(5, 3.2))
plt.bar(["0: 납기준수(Y)", "1: 납기지연(N)"], cnt.values, color=["#4e79a7", "#e15759"])
for i, v in enumerate(cnt.values):
    plt.text(i, v, f"{v}건 ({v/len(y)*100:.1f}%)", ha="center", va="bottom")
plt.ylim(0, cnt.values.max() * 1.2)
plt.title("목표변수 분포")
plt.show()
```

```
on_time_completion_flag
N    1337
Y     263

양성(1) 비율            : 83.56%
'전부 음성' 모델의 정확도 : 16.44%  ← 이 숫자를 넘어야 의미가 있다
```



---

6. 탐색 ① 그룹별 양성률 — 어느 조건에서 문제가 많은가

반드시 표본 수를 함께 보세요. "10건 중 2건(20%)"과 "400건 중 40건(10%)"은 비율만 보면 앞이 2배 높지만, 앞은 1건만 달라져도 크게 흔들립니다.

```python
def group_rate(col):
    g = df.groupby(col)["target"]
    t = pd.DataFrame({"전체건수": g.size(), "양성건수": g.sum().astype(int)})
    t["양성률(%)"] = (t["양성건수"] / t["전체건수"] * 100).round(2)
    return t


GROUPS = ["bottleneck_station", "line_id", "product_family", "order_priority"]
for c in GROUPS:
    print(f"===== {c} =====")
    display(group_rate(c))
```

```
===== bottleneck_station =====
```

bottleneck_station	전체건수	양성건수	양성률(%)
A	230	172	74.780
B	669	547	81.760
C	659	583	88.470
D	42	35	83.330

```
===== line_id =====
```

line_id	전체건수	양성건수	양성률(%)
L1	558	465	83.330
L2	519	427	82.270
L3	523	445	85.090

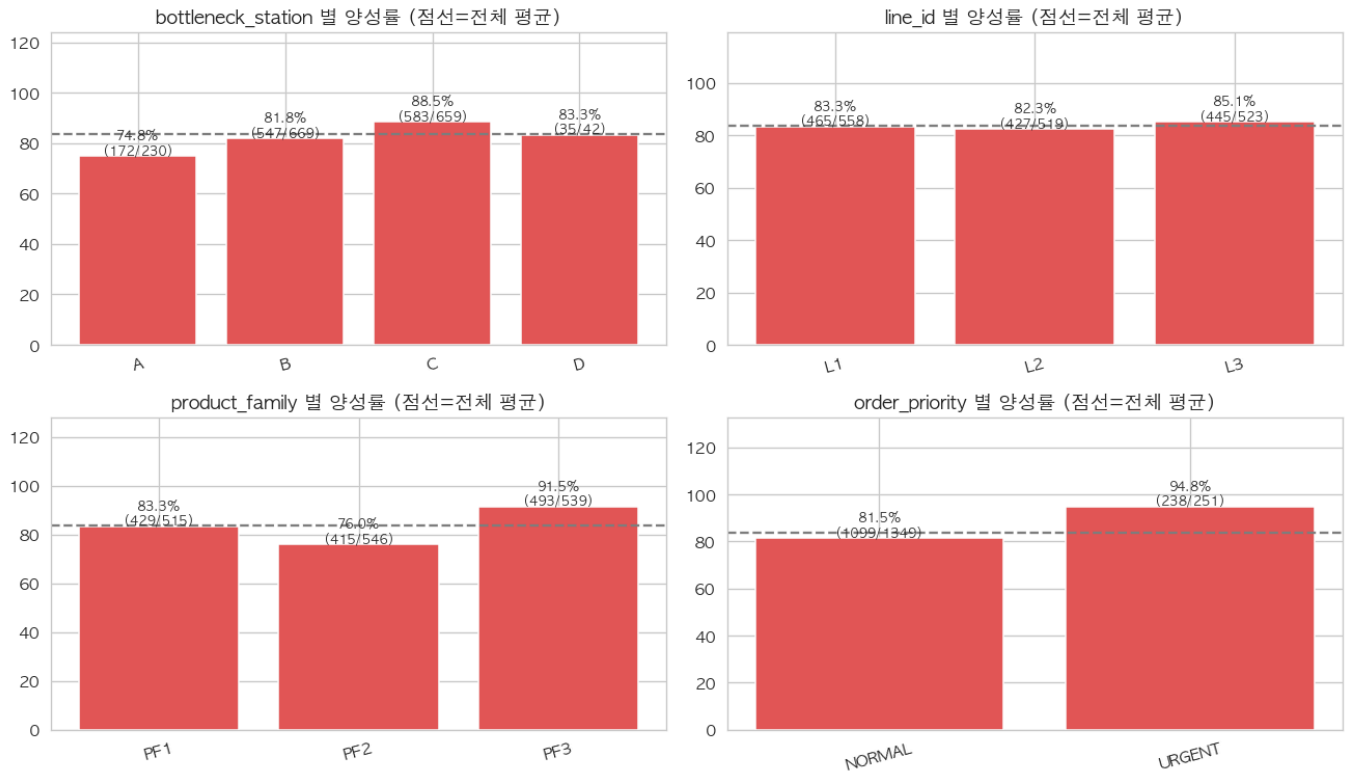
```
===== product_family =====
```

product_family	전체건수	양성건수	양성률(%)
PF1	515	429	83.300
PF2	546	415	76.010
PF3	539	493	91.470

```
===== order_priority =====
```

order_priority	전체건수	양성건수	양성률(%)
NORMAL	1349	1099	81.470
URGENT	251	238	94.820

```
overall = df["target"].mean() * 100
fig, axes = plt.subplots(2, 2, figsize=(12, 7))
for ax, c in zip(axes.ravel(), GROUPS):
    t = group_rate(c)
    ax.bar(t.index.astype(str), t["양성률(%)"], color="#e15759")
    ax.axhline(overall, color="gray", ls="--")
    for i, (n, m, r) in enumerate(zip(t["전체건수"], t["양성건수"], t["양성률(%)"])):
        ax.text(i, r, f"{r:.1f}%\n({m}/{n})", ha="center", va="bottom", fontsize=8)
    ax.set_title(f"{c} 별 양성률 (점선=전체 평균)")
    ax.set_ylim(0, max(t["양성률(%)"].max() * 1.4, 1))
    ax.tick_params(axis="x", rotation=15)
plt.tight_layout()
plt.show()
```



---

7. 탐색 ② 교차표 — 두 조건을 겹쳐서 보기

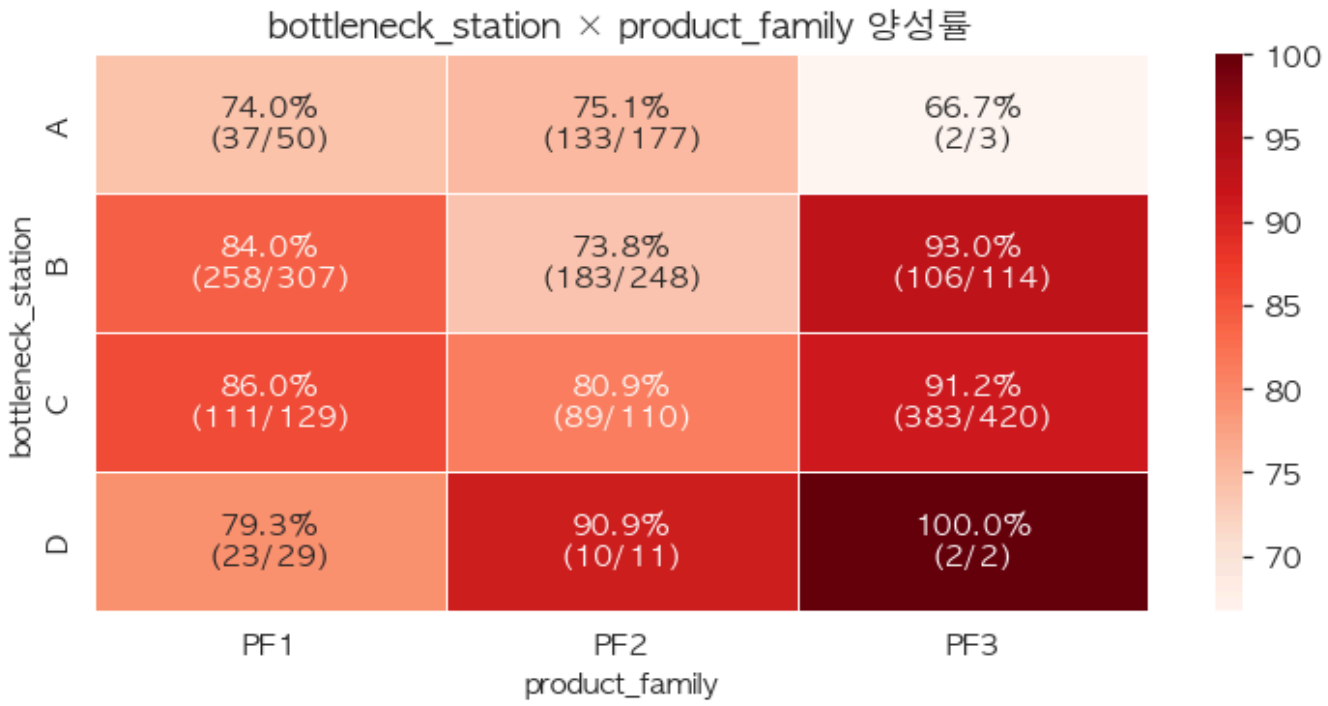
그룹을 하나씩 볼 때 보이는 차이가, 사실은 **다른 변수 때문**일 수 있습니다(**교란**). `bottleneck_station` 와 `product_family` 를 겹쳐서 확인합니다.

```
GX, GY = "bottleneck_station", "product_family"

cnt = pd.crosstab(df[GX], df[GY])
pos = pd.crosstab(df[GX], df[GY], values=df["target"], aggfunc="sum").fillna(0)
rate = (pos / cnt * 100).round(1)

plt.figure(figsize=(8, 3.4))
annot = rate.astype(str) + "%\n(" + pos.astype(int).astype(str) + "/" + cnt.astype(str) + ")"
sns.heatmap(rate, annot=annot, fmt="", cmap="Reds", linewidths=0.5)
plt.title(f"{GX} × {GY} 양성률")
plt.show()

print("칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.")
```




```
칸의 건수가 적으면(10건 미만) 그 칸의 비율은 참고만 한다.
```

---

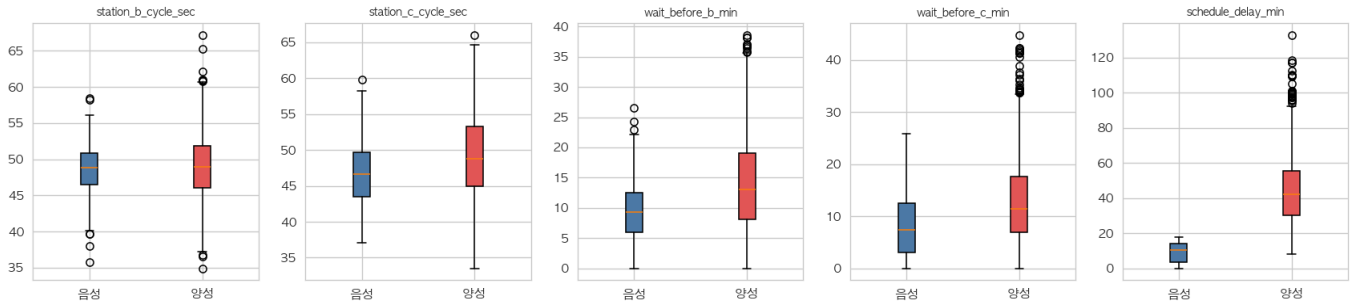
8. 탐색 ③ 수치형 변수 비교 — 양성/음성은 무엇이 다른가

상자가 많이 겹치면 **그 변수 하나로는 구분할 수 없다**는 뜻입니다. 제조 데이터는 대부분 이렇고, 그래서 **여러 변수를 조합하는 머신러닝**이 필요합니다.

```python
NUM_VIZ = [c for c in [
    "station_b_cycle_sec", "station_c_cycle_sec", "wait_before_b_min", "wait_before_c_min",
    "schedule_delay_min"
] if c in df.columns]

fig, axes = plt.subplots(1, len(NUM_VIZ), figsize=(3.1 * len(NUM_VIZ), 3.6))
for ax, c in zip(np.atleast_1d(axes).ravel(), NUM_VIZ):
    data = [df.loc[df["target"] == 0, c].dropna(), df.loc[df["target"] == 1, c].dropna()]
    bp = ax.boxplot(data, patch_artist=True)
    for patch, color in zip(bp["boxes"], ["#4e79a7", "#e15759"]):
        patch.set_facecolor(color)
    ax.set_xticks([1, 2])
    ax.set_xticklabels(["음성", "양성"])
    ax.set_title(c, fontsize=9)
plt.tight_layout()
plt.show()

cmp = pd.DataFrame({"음성 평균": df[df["target"] == 0][NUM_VIZ].mean(),
                    "양성 평균": df[df["target"] == 1][NUM_VIZ].mean()})
cmp["차이"] = cmp["양성 평균"] - cmp["음성 평균"]
display(cmp.round(3))
```



	음성 평균	양성 평균	차이
station_b_cycle_sec	48.474	49.050	0.576
station_c_cycle_sec	46.904	49.001	2.097
wait_before_b_min	9.633	13.830	4.197
wait_before_c_min	7.937	12.746	4.810
schedule_delay_min	8.996	44.867	35.871

---

9. 탐색 ④ 산점도와 상관관계

산점도는 점 하나가 기록 하나입니다. 색으로 양성/음성을 구분하면 **문제가 어느 영역에 모여 있는지** 눈으로 확인할 수 있습니다.

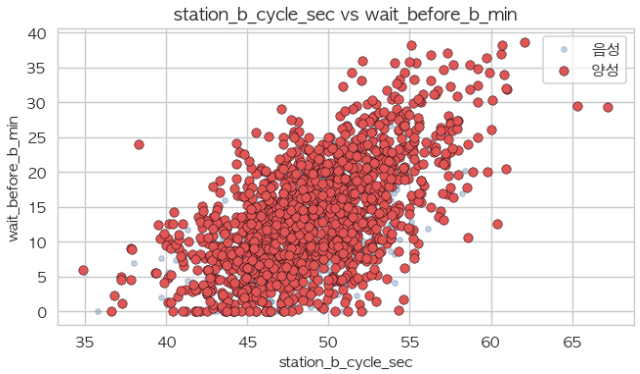
> B공정이 느릴 때 그 앞 대기가 길어지는가 - 병목의 전형적 신호

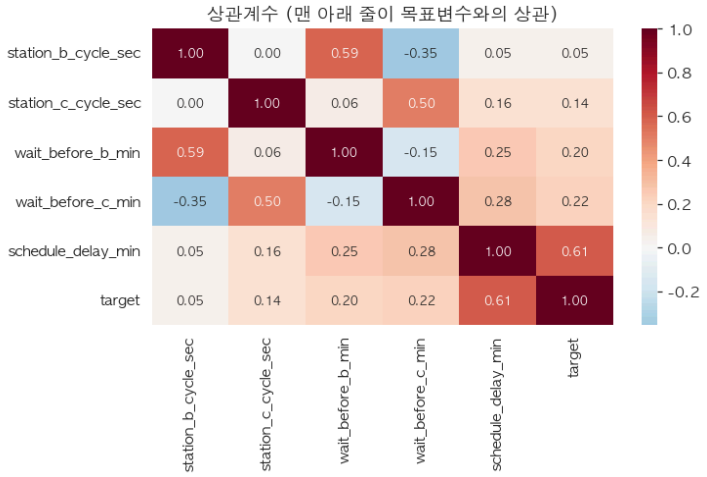
상관계수는 **직선 관계의 강도**(-1~+1)입니다. 세 가지를 기억하세요: 곡선 관계는 못 잡고, **상관은 인과가 아니며**, 이상치에 민감합니다.

```python
X1, Y1 = "station_b_cycle_sec", "wait_before_b_min"

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))
neg, pos_ = df[df["target"] == 0], df[df["target"] == 1]
axes[0].scatter(neg[X1], neg[Y1], s=12, alpha=0.3, c="#4e79a7", label="음성")
axes[0].scatter(pos_[X1], pos_[Y1], s=38, c="#e15759", edgecolors="black",
                linewidths=0.3, label="양성")
axes[0].set_xlabel(X1)
axes[0].set_ylabel(Y1)
axes[0].set_title(f"{X1} vs {Y1}")
axes[0].legend()

corr = df[NUM_VIZ + ["target"]].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=axes[1],
            annot_kws={"size": 8})
axes[1].set_title("상관계수 (맨 아래 줄이 목표변수와의 상관)")
plt.tight_layout()
plt.show()
```





---

10. 입력변수 고르기 — 데이터 누출 막기

판단 시점(교대 시작 전 (계획수량·제품군·투입 작업자 수·교체계획은 알고, 실제 사이클타임·대기시간·완료수량은 모름)) 에 **이미 알 수 있는 변수만** 사용합니다.

쓸 수 없는 변수 (결과가 나온 뒤에 생기는 값)

변수	이유
`actual_qty`	실제 완료수량. 생산이 끝나야 안다
`wip_end_qty`	종료 WIP. 로트 종료 시점의 결과다
`actual_lead_time_min`	실제 리드타임. 사실상 지연 여부와 같은 정보다
`bottleneck_station`	병목공정. 로트가 끝난 뒤 판정된 결과다(단, 별도 분류문제의 정답으로는 사용)
`schedule_delay_min`	일정지연(분). 납기준수 여부를 계산한 근거값이다 → 가장 치명적인 누출
`record_id`	단순 식별번호

> 성능이 비현실적으로 높으면(정확도 99%, AUC 0.99) **먼저 누출을 의심하세요.** 실무에서 가장 흔한 실패 원인입니다.

```
CAT = [c for c in ["shift", "line_id", "product_family", "order_priority"] if c in df.columns
NUM = [c for c in [
    "planned_qty", "available_operators", "changeover_min", "wip_start_qty",
    "planned_lead_time_min"
] if c in df.columns]      # 수치형 입력
LEAK = [
    "actual_qty", "wip_end_qty", "actual_lead_time_min", "bottleneck_station",
    "schedule_delay_min", "record_id"
]                                    # 사용 금지(데이터 누출)

print(f"입력 변수: 범주형 {len(CAT)}개 + 수치형 {len(NUM)}개 = {len(CAT)+len(NUM)}개")
print("범주형:", CAT)
print("수치형:", NUM)
```

```
입력 변수: 범주형 4개 + 수치형 5개 = 9개
범주형: ['shift', 'line_id', 'product_family', 'order_priority']
수치형: ['planned_qty', 'available_operators', 'changeover_min', 'wip_start_qty', 'planned_lead_
time_min']
```

11. 전처리 — 원핫인코딩 · 분리 · 스케일링

처리	이유
원핫인코딩	모델은 문자를 모른다. 범주마다 0/1 열을 만든다
학습/평가 분리	학습에 쓴 데이터로 평가하면 항상 성적이 좋다(시험문제를 미리 본 셈)
스케일링	변수마다 단위·범위가 달라서 큰 숫자 변수가 과대평가된다

순서가 중요합니다. 스케일러는 **학습 데이터에서만** 기준(평균·표준편차)을 배우고, 평가 데이터에는 그 기준을 **적용만** 해야 합니다. 전체 데이터로 먼저 스케일링하면 평가 데이터의 정보가 학습에 새어 들어갑니다(**데이터 누출**).

```
# ① 원핫인코딩 + 입력행렬 만들기
X = pd.concat([df[NUM], pd.get_dummies(df[CAT])], axis=1)
print("원핫인코딩 후 열 개수:", X.shape[1])

# ② 학습 75% / 평가 25% 로 분리 (stratify=y 로 양성 비율을 양쪽에 똑같이 맞춘다)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# ③ 스케일링 – 반드시 학습 데이터에서만 fit !
sc = StandardScaler()
X_train = X_train.copy()
X_test = X_test.copy()
X_train[NUM] = sc.fit_transform(X_train[NUM])   # 학습: 기준을 배우고 변환
X_test[NUM] = sc.transform(X_test[NUM])         # 평가: 배운 기준으로 변환만

print(f"학습 {len(X_train)}건 (양성 {int(y_train.sum())}) / "
      f"평가 {len(X_test)}건 (양성 {int(y_test.sum())})")
X_train.head(3)
```

```
원핫인코딩 후 열 개수: 15
학습 1200건 (양성 1003) / 평가 400건 (양성 334)
```

	planned_qty	available_operators	changeover_min	wip_start_qty	planned_lead_time_min	shift_Day	s
1263	0.180	0.293	-0.499	0.377	-0.885	False	
878	1.090	-0.386	-0.499	-0.522	1.529	True	
616	-0.385	0.293	1.347	-0.896	-0.846	True	

---

12. 모델 3종 학습과 성능평가

모델	방식	특징
로지스틱 회귀	변수에 가중치를 곱해 더한 뒤 확률로 변환	해석이 쉽다
의사결정나무	스무고개처럼 질문을 이어가며 나눔	그림으로 볼 수 있다
랜덤포레스트	나무 300그루의 다수결	성능이 가장 좋은 편

`class_weight="balanced"` **를 반드시 넣으세요.** 양성이 드물면 모델은 "전부 음성"이라고 답하는 것이 손해가 적다고 판단합니다. 이 옵션은 **드문 쪽을 틀렸을 때 벌점을 크게** 주어 그것을 막습니다.

혼동행렬과 지표

	예측: 음성	예측: 양성
실제 음성	TN	**FP (헛경보)**
실제 양성	**FN (놓침)**	TP

- **Recall** = TP/(TP+FN) : 실제 양성 중 몇 %를 잡았나 → **놓치면 안 될 때 중요**
- **Precision** = TP/(TP+FP) : 경보 중 몇 %가 진짜였나 → **헛경보 비용이 클 때 중요**
- **ROC-AUC** : 임계값과 무관하게 "양성에 더 높은 점수를 줄 확률". 0.5=무작위

```python
models = {
    "로지스틱회귀": LogisticRegression(max_iter=2000, class_weight="balanced"),
    "의사결정나무": DecisionTreeClassifier(max_depth=4, min_samples_leaf=20,
                                          class_weight="balanced", random_state=42),
    "랜덤포레스트": RandomForestClassifier(n_estimators=300, max_depth=8,
                                          min_samples_leaf=5, class_weight="balanced",
                                          random_state=42),
}

rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    proba = m.predict_proba(X_test)[:, 1]
    pred = m.predict(X_test)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"모델": name, "TP": tp, "FN": fn, "FP": fp, "TN": tn,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0),
                 "ROC_AUC": roc_auc_score(y_test, proba)})
    print(f"===== {name} =====")
    print(classification_report(y_test, pred, zero_division=0))

result = pd.DataFrame(rows).set_index("모델")
display(result.round(3))

base = 1 - y_test.mean()
print(f"참고: '전부 음성' 모델의 정확도 = {base*100:.1f}% (이 숫자는 Recall 0 이다)")
```

```
===== 로지스틱회귀 =====
              precision    recall  f1-score   support

           0       0.36      0.77      0.49        66
           1       0.94      0.72      0.82       334

    accuracy                           0.73       400
   macro avg       0.65      0.75      0.65       400
weighted avg       0.85      0.73      0.76       400

===== 의사결정나무 =====
              precision    recall  f1-score   support

           0       0.35      0.65      0.46        66
           1       0.92      0.76      0.83       334

    accuracy                           0.74       400
   macro avg       0.63      0.71      0.65       400
weighted avg       0.82      0.74      0.77       400

===== 랜덤포레스트 =====
              precision    recall  f1-score   support

           0       0.39      0.59      0.47        66
           1       0.91      0.82      0.86       334

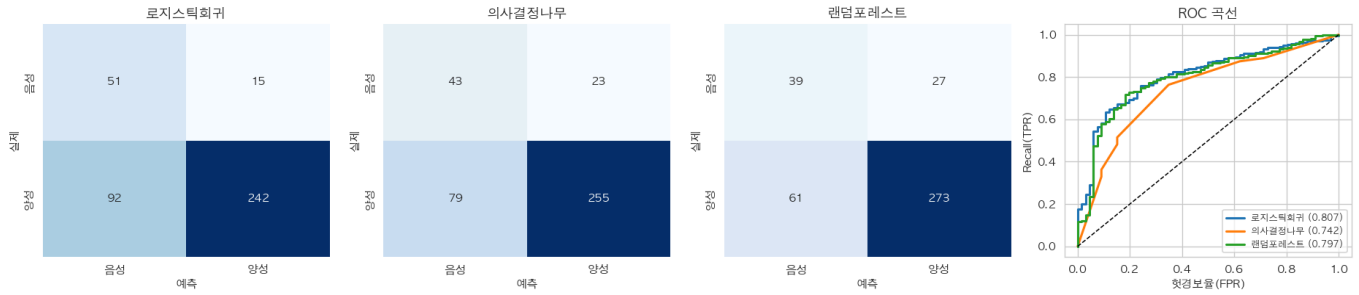
    accuracy                           0.78       400
   macro avg       0.65      0.70      0.67       400
weighted avg       0.82      0.78      0.80       400
```

모델	TP	FN	FP	TN	Recall	Precision	ROC_AUC
로지스틱회귀	242	92	15	51	0.725	0.942	0.807
의사결정나무	255	79	23	43	0.763	0.917	0.742
랜덤포레스트	273	61	27	39	0.817	0.910	0.797

참고: '전부 음성' 모델의 정확도 = 16.5% (이 숫자는 Recall 0 이다)

```
# 혼동행렬 + ROC 곡선
fig, axes = plt.subplots(1, 4, figsize=(17, 3.8))
for ax, (name, m) in zip(axes[:3], models.items()):
    cm = confusion_matrix(y_test, m.predict(X_test), labels=[0, 1])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=ax,
                xticklabels=["음성", "양성"], yticklabels=["음성", "양성"])
    ax.set_title(name)
    ax.set_xlabel("예측")
    ax.set_ylabel("실제")

for name, m in models.items():
    proba = m.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, proba)
    axes[3].plot(fpr, tpr, lw=2, label=f"{name} ({roc_auc_score(y_test, proba):.3f})")
axes[3].plot([0, 1], [0, 1], "k--", lw=1)
axes[3].set_xlabel("헛경보율(FPR)")
axes[3].set_ylabel("Recall(TPR)")
axes[3].set_title("ROC 곡선")
axes[3].legend(fontsize=8)
plt.tight_layout()
plt.show()
```



---

13. 임계값 조정 — 모델을 바꾸지 않고 운영점 바꾸기

모델은 0/1이 아니라 **확률**을 출력합니다. `predict()` 는 0.5를 기준으로 자르는데, 이 기준은 **바꿀 수 있습니다.**

- 임계값을 **낮추면**: 경보가 늘고 → Recall↑, Precision↓
- 임계값을 **높이면**: 경보가 줄고 → Recall↓, Precision↑

현장 제약을 먼저 정하고 임계값을 고르는 것이 올바른 순서입니다.

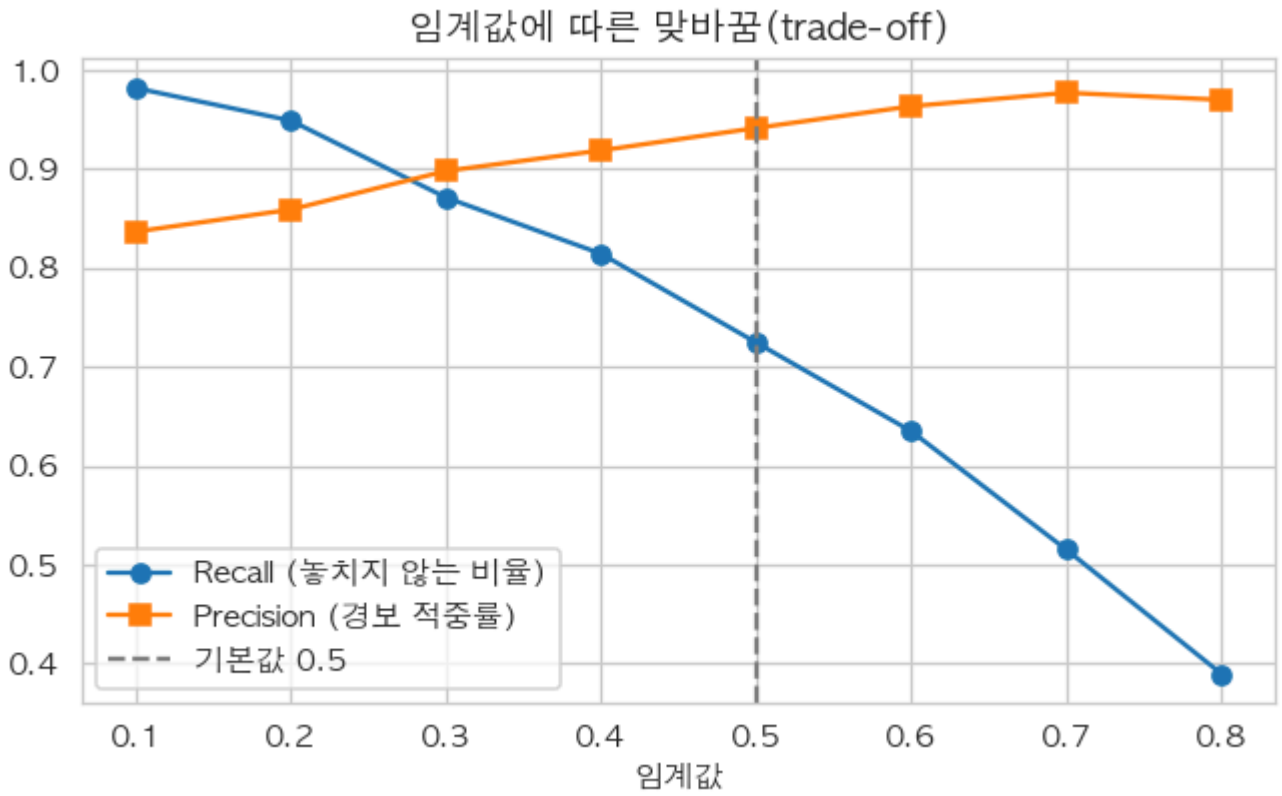
```python
best_name = result["ROC_AUC"].idxmax()          # 임계값과 무관한 지표로 기준 모델 선택
best = models[best_name]
proba = best.predict_proba(X_test)[:, 1]
print("기준 모델:", best_name)

rows = []
for th in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    pred = (proba >= th).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_test, pred, labels=[0, 1]).ravel()
    rows.append({"임계값": th, "경보건수": int(pred.sum()), "TP(잡음)": tp,
                 "FN(놓침)": fn, "FP(헛경보)": fp,
                 "Recall": recall_score(y_test, pred, zero_division=0),
                 "Precision": precision_score(y_test, pred, zero_division=0)})
th_tbl = pd.DataFrame(rows)
display(th_tbl.round(3))

plt.figure(figsize=(7, 3.8))
plt.plot(th_tbl["임계값"], th_tbl["Recall"], "o-", label="Recall (놓치지 않는 비율)")
plt.plot(th_tbl["임계값"], th_tbl["Precision"], "s-", label="Precision (경보 적중률)")
plt.axvline(0.5, color="gray", ls="--", label="기본값 0.5")
plt.xlabel("임계값")
plt.legend()
plt.title("임계값에 따른 맞바꿈(trade-off)")
plt.show()
```

```
기준 모델: 로지스틱회귀
```

	임계값	경보건수	TP(잡음)	FN(놓침)	FP(헛경보)	Recall	Precision
0	0.100	392	328	6	64	0.982	0.837
1	0.200	369	317	17	52	0.949	0.859
2	0.300	324	291	43	33	0.871	0.898
3	0.400	296	272	62	24	0.814	0.919
4	0.500	257	242	92	15	0.725	0.942
5	0.600	220	212	122	8	0.635	0.964
6	0.700	176	172	162	4	0.515	0.977
7	0.800	134	130	204	4	0.389	0.970



---

14. 특성중요도 — 모델은 무엇을 보고 판단했나

랜덤포레스트는 "이 변수로 나눴을 때 양성/음성이 얼마나 깔끔하게 갈리는가"를 기준으로 분기합니다. 그 기여도를 모두 합한 것이 **특성중요도**입니다(합 = 1).

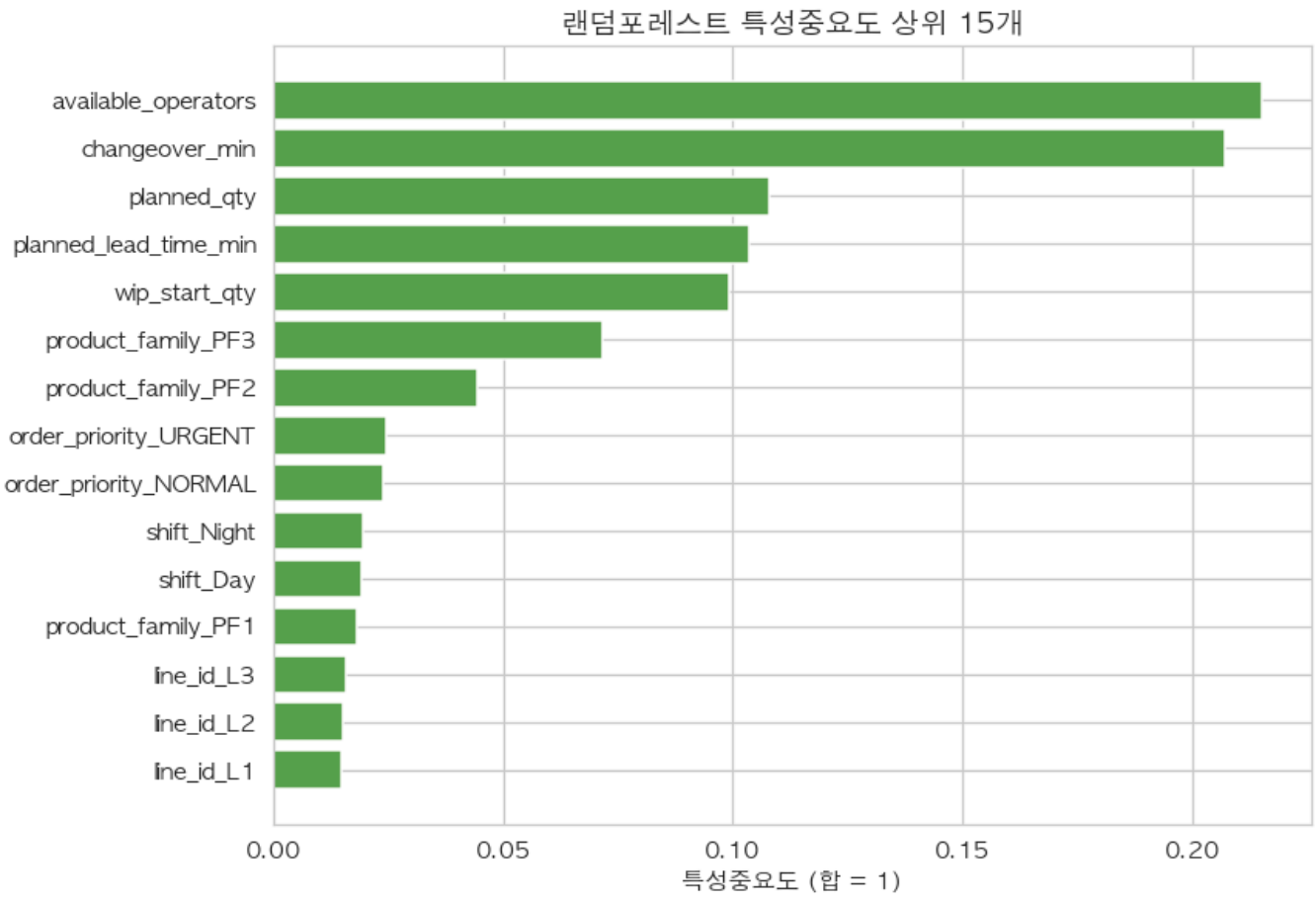
> **주의**: 특성중요도는 **"모델이 많이 참고한 변수"** 이지 **"현실의 원인"** 이 아닙니다. 인과를 주장하려면 현장에서 조건을 바꿔 보는 확인이 필요합니다.

```
rf = models["랜덤포레스트"]
imp = pd.DataFrame({"변수": X_train.columns,
                    "중요도": rf.feature_importances_}).sort_values("중요도", ascending=False)
display(imp.head(10).round(4))

top = imp.head(15)[::-1]
plt.figure(figsize=(8, 5.5))
plt.barh(top["변수"], top["중요도"], color="#59a14f")
plt.xlabel("특성중요도 (합 = 1)")
plt.title("랜덤포레스트 특성중요도 상위 15개")
plt.tight_layout()
plt.show()

print("상위 5개 변수가 전체 중요도의", f"{imp.head(5)['중요도'].sum()*100:.0f}% 를 차지합니다.")
```

	변수	중요도
1	available_operators	0.215
2	changeover_min	0.207
0	planned_qty	0.108
4	planned_lead_time_min	0.103
3	wip_start_qty	0.099
12	product_family_PF3	0.072
11	product_family_PF2	0.044
14	order_priority_URGENT	0.025
13	order_priority_NORMAL	0.024
6	shift_Night	0.020



상위 5개 변수가 전체 중요도의 73% 를 차지합니다.

---

15. 정리

이 데이터에서 확인한 것

- 병목 분포: **B 669건(41.8%), C 659건(41.2%)**, A 230건(14.4%), D 42건(2.6%). B와 C가 전체의 83%다 → 개선 투자는 B·C에 집중한다.
- 평균 계획 리드타임 209분 vs 실제 248분. **평균 39분씩 초과**한다.
- 종료 WIP 평균 43개 > 시작 WIP 평균 37개 → **전반적으로 재공품이 쌓이는 추세**다.
- PF1은 B공정, PF3은 C공정 병목이 상대적으로 많다 → **제품군별로 개선 대상이 다르다.**
- C공정 병목의 평균 일정지연이 약 42.6분으로 가장 크다.
- 납기준수율 16.4%는 매우 낮다. 모델을 만드는 것과 별개로 **계획 리드타임 산정 기준 재검토**를 제안해야 한다.
- 결측: C공정 사이클타임 10건, C공정 전 대기 16건. 공교롭게 **문제 공정에 결측이 있다**. 결측을 버리면 병목 분석이 왜곡될 수 있다.
- 실제완료수량이 계획수량을 초과하는 기록은 없다(논리적 정합성 OK).

결론과 다음 단계

- 납기지연이 83.6%다. "전부 지연"이라고 답해도 정확도 83.6%이므로 **정시 완료(16.4%)를 구분해 내는 능력**을 봐야 한다.
- 교대 시작 전 정보만으로는 예측이 어렵다. 이 사실 자체가 **"공정 실적을 실시간으로 수집해야 한다"** 는 개선 제안의 근거가 된다.
- 지연률이 이렇게 높다는 것은 **계획 리드타임 산정 기준** 자체를 재검토해야 한다는 신호다.
- 다음 단계: 병목공정(A/B/C/D)을 맞히는 **다중분류**와, 공정별 대기시간을 이용한 원인분석.

공통으로 기억할 5가지

#	원칙
1	**정확도만 보지 않는다** — 양성이 드물면 아무것도 안 해도 95%가 넘는다
2	**놓침(FN)과 헛경보(FP)의 비용은 다르다** — 어느 쪽이 비싼지는 현장이 정한다
3	**성능이 너무 좋으면 누출을 의심한다** — 결과 변수를 넣지 않았는지 확인
4	**임계값은 운영 정책이다** — 모델을 바꾸지 않고도 경보 수를 조절할 수 있다
5	**상관은 인과가 아니다** — 특성중요도는 원인이 아니라 단서다

연습문제 / 생각해 볼 질문

아래 문항은 정답을 맞히는 시험이 아니라, **발표자료에 쓸 문장을 만드는 연습**입니다. 코드를 조금씩 바꿔 보고, 결과를 한 문장으로 정리해 보세요.

1. 공정별 사이클타임 4개의 평균·중앙값·P90을 한 표로 만들고, 어느 공정이 '평소에도 느린지'와 '가끔 폭발하는지'를 구분하세요.
2. `bottleneck_station` 별로 공정 사이클타임 4개의 평균을 비교하는 표를 만드세요. B병목 로트는 정말 B가 느린가요?
3. 세트 A(사전예측)와 세트 B(사후분석) 모델의 Recall·AUC를 비교하고, 교대 시작 전 예측이 얼마나 어려운지 수치로 설명하세요.
4. `schedule_delay_min` 을 특성에 넣어 보세요. AUC가 1.0이 되는 것을 확인하고 왜 쓸 수 없는지 쓰세요.